Phase 1 - Grade 4 - Vectorized Ops

Key Learning Points

Boolean masking with a single condition is extended to MULTIPLE conditions at once -- e.g. 'stock > 0 AND price < 20' -- plus .apply(), an escape hatch for logic too custom for a simple comparison.

& (elementwise AND)
Combines two boolean arrays/columns element-by-element. Must replace Python's 'and' when working with pandas/NumPy columns; each side MUST be parenthesized due to operator precedence.

| (elementwise OR)
Same idea as & but for OR logic, elementwise across a whole column/array.

~ (elementwise NOT)
Flips True/False elementwise -- vectorized equivalent of Python's 'not'.

.apply(func)
Runs a custom function once per element in a Series -- for logic too complex for a comparison or &/| combination. Slower (a loop under the hood again) but flexible.

Operator precedence trap
Python evaluates & BEFORE >/< unless parentheses force the comparisons first. Omitting parentheses around each condition silently breaks the expression.

Why plain and/or break on columns

Python's 'and' expects exactly ONE True/False value on each side. But df['stock'] > 0 is not one value -- it's an entire column of True/False values, one per row. Python has no defined way to 'and' two whole columns with the plain keyword, so it errors. '&' is built specifically to compare two arrays element-by-element.

Examples

```
data = {'product': ['Pen','Notebook','Eraser','Marker'],
        'price':   [10, 40, 5, 25],
        'stock':   [100, 30, 200, 0]}
df = pd.DataFrame(data)

df[(df['price'] < 30) & (df['stock'] > 50)]     # AND -- parentheses mandatory on both sides
df[(df['price'] < 10) | (df['stock'] == 0)]     # OR

def stock_logic(s):
    return "In Stock" if s > 0 else "Out of Stock"
df['stock_status'] = df['stock'].apply(stock_logic)

df[~(df['stock'] > 50)]      # NOT -- rows where stock is NOT > 50
```

Quiz Q&A;

Q1. Get rows where price ≥ 25 AND stock < 100.

A: df[(df['price'] >= 25) & (df['stock'] < 100)] -- BOTH conditions individually parenthesized.

Q2. Why does df[df['price'] > 20 and df['stock'] > 0] throw an error?

A: 'and' expects a single True/False value, but each comparison produces a whole column of True/False values. Python can't apply a plain 'and' to two entire columns at once, so it errors.

Q3. Using .apply(), create price_tier = 'Premium' if price > 20 else 'Budget'.

A: def tier(s): return 'Premium' if s > 20 else 'Budget' df['price_tier'] = df['price'].apply(tier)

Q4. Get rows where stock is NOT greater than 50, using ~.

A: df[~(df['stock'] > 50)]

Q5. True or False: '&' and 'and' behave identically in pandas, just with different symbols.

A: False. 'and' is for a single boolean value; '&' is for elementwise comparison across whole columns/arrays. Using 'and' on a column raises an error.